

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Quality Guidelines

These quality control guidelines define the quality bar for TBench Edition 2 formatted tasks. All tasks must comply with these standards to pass review.

High Level Requirements

All tasks must align with these essential tenants:

Instruction Prompt Styling: Each task will be equipped with an `instruction.md` which outlines the task to be completed. These task prompts will describe the problem in 1-2 paragraphs and the requirements in no more than 2 paragraphs of 20 bullets. Prompts should not be LLM generated and should read in a way that is reflective of how users interact with coding agents.

Multi-Step: Tasks should require chaining multiple commands (5 minimum terminal commands), handling intermediate states, and some reasoning (like error recovery or branching), and avoid being solvable with a single command or a single episode of commands.

Testable: Each task must be fully specified and self-contained for the agent to solve without ambiguity and should be accompanied by a sufficient set of tests that can be used to deterministically measure the final state of the environment and determine if the task has been completed correctly.

Novel: Avoid ANY variations of existing tasks in TerminalBench Repository; each task should introduce a new task setup, definition, and data used to solve the task. Tasks should also be novel with respect to tasks included in Snorkel's Terminal Bench Edition 1 dataset.

No Privileged Ops: Tasks must not require root-level privileges or unsafe Docker settings like `--privileged`.

Standalone: The task must run to completion without human input after start; all parameters are provided via files, flags, or environment variables. To validate this, we run and validate all tasks using the harbor framework. Single-container tasks are validated in Daytona, while multi-container tasks are validated in a Docker environment. We use CI/CD infrastructure to support the running of multi-container tasks at scale.

Specific Scenarios to Avoid

1. No Latency-Based Tests

Rule: Tests should not check for improved latency or performance metrics.

Why: Latency and performance tests are hardware-dependent. Oracle solutions may not be reproducible when run on different hardware, causing tasks to pass on some machines and fail on others.

Example of what to avoid:

PYTHON

 Copy

```
# ❌ Bad - From test_outputs.py
def test_latency_values_reasonable():
    """Check that latency values are reasonable (not negative or ex
    with open("/app/perf/results.json") as f:
        data = json.load(f)

    baseline_perf = data.get("baseline", {}).get("performance", {})
    optimized_perf = data.get("optimized", {}).get("performance", {})

    assert p50 <= 10000, f"{name} p50 latency ({p50}ms) seems high"
    assert p95 <= 10000, f"{name} p95 latency ({p95}ms) seems high"
    assert p99 <= 10000, f"{name} p99 latency ({p99}ms) seems high"
```

Instead: Test for correctness and functionality, not performance characteristics.

2. Identical Testing for Oracle and Agent

Rule: The testing logic must be exactly the same for both the oracle solution and agents. Conditional logic that only runs for either oracle or agent is explicitly banned.

Why: Different testing conditions make tasks unfair and potentially unsolvable for agents.

Example of what to avoid:

BASH

 Copy

```
# ❌ Bad - From test.sh
# Detect oracle mode by checking for oracle directory
if [ -d "/oracle" ]; then
    export EVAL_IS_ORACLE=1
else
    export EVAL_IS_ORACLE=0
fi

if [ "$EVAL_IS_ORACLE" -eq 0 ]; then
    echo "Deny permissions for agent"
    chmod 000 eval/
fi
```

Instead: Apply the same permissions, environment, and test conditions to both oracle and agent.

3. Tag Docker-Compose and Multi-Container Tasks

Rule: Tasks using `docker-compose.yaml` or multiple containers must be tagged in `task.toml` metadata.

Required tags:

- `custom_docker_compose = true` — if the task contains a `docker-compose.yaml`
- `is_multi_container = true` — if the task uses multiple containers

Why: This allows tracking and filtering of tasks with these setups.

4. Avoid Fetching Data from the Web

Rule: Avoid including HTTP requests to web URLs within task environments or solutions.

Why:

- Webpages and API endpoints can change over time, rendering tasks unsolvable
- Network latency varies across environments, affecting agent and verifier timeouts

Exceptions: Package installs (pip, npm, apt, uv) are acceptable at image build time in the Dockerfile, but not at runtime in test.sh or solution scripts.

Example of what to avoid:

PYTHON

 Copy

```
# ❌ Bad - In solution or environment
URL = "https://database.lichess.org/standard/lichess_db_standard_ra

def main():
    if not os.path.exists(OUTPUT_DIR):
        os.makedirs(OUTPUT_DIR)

    dctx = zstd.ZstdDecompressor()

    print(f"Downloading stream from {URL}...")
    for attempt in range(5):
        try:
            with urllib.request.urlopen(URL) as response:
                with dctx.stream_reader(response) as reader:
                    # ... processing ...
```

Instead: Store small required data as files in `environment/`. Keep the build context within the Dockerfile size limits (100 MiB total, 50 MiB per file), and avoid baking large optional datasets into the image.

5. Don't Create `/tests` or `/solution` Directories

Rule: Do not create or modify `/tests` or `/solution` directories in your Dockerfile.

Why: The Harbor framework reserves these directories for running the oracle solution and tests. Creating or modifying them causes conflicts with the framework.

Examples of what to avoid:

DOCKERFILE

[Copy](#)

```
# ❌ Bad - Changes permissions on reserved directories
RUN chown -R appuser:appuser /app/output /app/logs /oracle /tests

# ❌ Bad - Creates reserved directories
RUN mkdir -p /tests
RUN mkdir -p /oracle
```

Instead: Use different directory names for your task's internal files.

6. Always Write Reward File on Failure

Rule: Failed agent runs must write `0` to the reward file, not exit before writing.

Why: The Harbor harness expects a `reward.txt` file. Exiting before writing causes `RewardFileNotFound` errors.

Correct pattern:

BASH

 Copy

```
#  Good – Always writes reward file
python -m pytest /tests/test_outputs.py -rA

if [ $? -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt
else
    echo 0 > /logs/verifier/reward.txt
fi
```

Example of what to avoid:

BASH

 Copy

```
#  Bad – Exits before writing reward file
python -m pytest /tests/test_outputs.py -rA

exit_code=$?

if [ $exit_code -eq 0 ]; then
    echo "All tests passed successfully!"
else
    echo "Tests failed with exit code: $exit_code"
fi

exit $exit_code # ← Exits here, reward file never written!

if [ $? -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt
else
    echo 0 > /logs/verifier/reward.txt
fi
```

7. Provide Default Values for Environment Variables

Rule: Environment variables used in `test.sh` must have default values compatible with the task structure.

Why: Tasks may be run in frameworks other than Harbor where certain environment variables aren't set.

Exception: Standard environment variables like `$HOME` and `$PWD` don't need defaults.

Most common case: `TEST_DIR` should default to `/tests`:

BASH

 Copy

```
TEST_DIR="${TEST_DIR:-/tests}"
```

Good examples:

BASH

 Copy

```
#  Good - Variable with default  
TEST_DIR="${TEST_DIR:-/tests}"  
python -m pytest "$TEST_DIR/test_outputs.py" -rA -v
```

BASH

 Copy

```
#  Good - Inline default  
pytest ${TEST_DIR:-/tests}/test_outputs.py -rA
```

BASH

 Copy

```
#  Good - Hardcoded path (also acceptable)  
pytest /tests/test_outputs.py -rA
```

Bad examples:

BASH

 Copy

```
#  Bad - Fails if TEST_DIR not set  
if [ -z "${TEST_DIR:-}" ]; then
```

```
    echo "TEST_DIR environment variable is not set." >&2
    exit 1
fi
```

BASH

 Copy

```
# ✗ Bad – No default provided
python -m pytest "$TEST_DIR/test_outputs.py" -rA -v
```

8. Avoid Oracle-Replication Performance Thresholds

Rule: Performance thresholds must not be set so tightly relative to the oracle (e.g., within ~5% of oracle performance) that the task effectively becomes “replicate the oracle.” Thresholds should evaluate whether the agent meaningfully solves the problem — not whether it nearly matches a specific implementation.

Why: Extremely tight margins turn optimization tasks into imitation exercises rather than problem-solving evaluations. This discourages diverse, valid solution strategies and rewards oracle mimicry over genuine capability.

Example of what to avoid:

BASH

 Copy

```
# ✗ Bad – Threshold set to near-oracle performance

oracle_score = 0.92
required_score = oracle_score * 0.95    # ~5% margin

agent_score = evaluate_agent()

assert agent_score >= required_score
```

This structure effectively requires the agent to nearly replicate the oracle’s performance rather than demonstrate a strong, valid solution.

Instead:

- Set thresholds that reflect meaningful task completion, not oracle proximity.
- Ensure multiple solution strategies could reasonably pass.
- Validate that the task measures true problem-solving ability rather than implementation similarity.
- Ask: “Would a fundamentally different but correct approach be able to pass this threshold?”
 - If the answer is no, the threshold is likely too tight and should be loosened.

Quick Reference

RULE	SUMMARY
No latency tests	Don't test for performance/timing metrics
Identical testing	Same conditions for oracle and agent
Tag docker-compose	Add <code>custom_docker_compose = true</code> to metadata
Tag multi-container	Add <code>is_multi_container = true</code> to metadata
No web fetching	Store data locally, don't download at runtime
Reserved directories	Don't create/modify <code>/tests</code> or <code>/solution</code>
Always write reward	Write <code>0</code> on failure, never exit early
Default env vars	Use <code>\${VAR:-default}</code> syntax for env vars

Next Steps

- [Common Errors](#) — More anti-patterns to avoid
- [CI Checks Reference](#) — Automated check details
- [Task Requirements](#) — Complete requirements list

PREVIOUS

[Defending Your Submission](#)

NEXT

[FAQ](#)